

Composition [Strong reference]

Composition in Java is a way to design classes such that one class contains an object of another class. It is a way of establishing a "HAS-A" relationship between classes.

Composition represents a strong relationship between the containing class and the contained class. If the containing object (Car object) is destroyed, all the contained objects (Engine object) are also destroyed.

A car has an engine. Composition makes strong relationship between the objects. It means that if we destroy the owner object, its members will be also destroyed with it. For example, if the Car is destroyed the engine will also be destroyed as well.

Program Guidelines :

- 1) One object cannot exist without another object
- 2) We will not create two separate objects in ELC class, during the creation of Car object, Engine object should be automatically created.
- 3) We can declare blank final field so, engine initialization is compulsory as well as we cannot change the Engine object.

//Program :

```
package com.ravi.composition;

public class Engine
{
    private String type;
    private int horsePower;

    public Engine(String type, int horsePower)
    {
        super();
        this.type = type;
        this.horsePower = horsePower;
    }

    @Override
    public String toString()
    {
        return "Engine [type=" + type + ", horsePower=" + horsePower + "];"
    }
}

package com.ravi.composition;

public class Car
{
    private int model;
    private String name;
    private final Engine engine; //HAS-A relation [Blank Final field]

    public Car(int model, String name)
    {
        this.model = model;
        this.name = name;
        this.engine = new Engine("Battery", 1200); //Composition
    }

    @Override
    public String toString()
    {
        return "Car [model=" + model + ", name=" + name + ", engine=" + engine + "];"
    }
}

package com.ravi.composition;

public class CompositionDemo {

    public static void main(String[] args)
    {
        Car c1 = new Car(2026, "Naxon");
        IO.println(c1);
    }
}
```

Aggregation (Weak Reference):

Aggregation in Java is another form of association between classes that represents a "HAS-A" relationship, but with a weaker bond compared to composition.

In aggregation, one class contains an object of another class, but the contained object can exist independently of the container. If the container object is destroyed, the contained object can still exist.

```
package com.ravi.aggregation;

public class Order
{
    private int id;
    private String foodName;
    private double foodPrice;
    private int quantity;

    public Order(int id, String foodName, double foodPrice, int quantity) {
        super();
        this.id = id;
        this.foodName = foodName;
        this.foodPrice = foodPrice;
        this.quantity = quantity;
    }

    @Override
    public String toString()
    {
        return "Order [id=" + id + ", foodName=" + foodName + ", foodPrice=" + foodPrice + ", quantity=" + quantity + "];"
    }
}

package com.ravi.aggregation;

public class Customer
{
    private int id;
    private String name;
    private String address;
    private long mobileNo;
    private Order order; //HAS-A relation

    public Customer(int id, String name, String address, long mobileNo, Order order)
    {
        super();
        this.id = id;
        this.name = name;
        this.address = address;
        this.mobileNo = mobileNo;
        this.order = order;
    }

    @Override
    public String toString()
    {
        return "Customer [id=" + id + ", name=" + name + ", address=" + address + ", mobileNo=" + mobileNo + ", order=" + order + "];"
    }
}

package com.ravi.aggregation;

public class AggregationDemo {

    public static void main(String[] args)
    {
        Order order = new Order(1, "Biryani", 450, 1);

        Customer c1 = new Customer(101, "Scott", "Ameerpet", 9812345678L, order);
        c1 = null;

        Customer c2 = new Customer(101, "John", "S R Nagar", 8712345678L, order);
        IO.println(c2);
    }
}
```

**** Method Overriding :

Re-defining the super class non static method with sub class non static method.
Not possible without Inheritance.
Method signature (Method name along with parameter) must be same.
return type must be compatible. (same type)
Method Overriding -> Non static methods
Method Hiding -> static methods

Re-defining the super class NON STATIC METHOD in the sub class in such a way that method signature (Method Name along with method parameter) must be same and return type must be compatible.

Method Overriding is not possible without inheritance.

Generally we can't change the return type of the method while overriding a method (compatibility issue) but from JDK 1.5v there is a concept called Co-variant (In same direction) through which we can achieve the compatible return type.

Example :

```
class Super
{
    public void m1() //Non static method
    {
    }
}

class Sub extends Super
{
    public void m1() //Overridden Method
    {
        //Here Redefining the super class method body
    }
}
```

Why we will override a method:

If we want to modify the super class non static method implementation, we want to provide our own implementation then we should override a method

Advantage of Method Overriding :

The advantage of Method Overriding is, each sub class is specifying its own specific behavior.

Example 1:

```
class Animal
{
    public void eat()
    {
        IO.println("Generic Animal is eating");
    }
}

class Dog extends Animal
{
    public void eat()
    {
        IO.println("Dog Animal is eating");
    }
}

class Horse extends Animal
{
    public void eat()
    {
        IO.println("Horse Animal is eating");
    }
}

Note : Dog and Horse classes are specifying its own specific behaviour
```

Example 2 :

```
class Payment
{
    public void makePayment()
    {
        IO.println("Making a generic payment");
    }
}

class UPI extends Payment
{
    public void makePayment()
    {
        IO.println("Making a payment through UPI");
    }
}

class DebitCard extends Payment
{
    public void makePayment()
    {
        IO.println("Making a payment through DebitCard");
    }
}

Note : UPI and DebitCard classes are specifying its own specific behaviour
```